

Iterazione 4

Durante l'iterazione 4 ci siamo concentrati sullo sviluppo dell'UC5, UC6 e UC7 e dei relativi test.

UC5: Segnalazione problemi struttura

Nome caso d'uso	UC5: Segnalazione problemi struttura
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Sportivo (Utente registrato)
Parti interessate e interessi	Sportivo: desidera segnalare malfunzionamenti o danni per garantire la qualità del servizio e la sicurezza durante l'attività.
Pre-condizioni	Lo sportivo ha effettuato il login al sistema.
Garanzie di successo	Una nuova istanza di "Segnalazione" viene creata, associata alla struttura e resa visibile all'amministratore.
Scenario principale di successo	1. Lo Sportivo accede alla sezione "Segnala Guasto". 2. Il Sistema richiede di selezionare la struttura coinvolta (es. Campo Tennis A). 3. Lo Sportivo seleziona la struttura e inserisce una descrizione dettagliata del problema (es. "Rete rotta"). 4. Lo Sportivo conferma l'invio della segnalazione. 5. Il Sistema registra la segnalazione, le assegna uno stato iniziale ("Aperta"), acquisisce la data corrente e la associa allo sportivo e alla struttura. 6. Il Sistema conferma allo Sportivo l'avvenuto invio.

Estensioni

***a.** In qualsiasi momento il sistema fallisce.

1. L'admin riavvia il software e ripristina lo stato.

3a. Lo Sportivo omette la descrizione del problema.

1. Il Sistema segnala che il campo descrizione è obbligatorio.

2. Lo Sportivo inserisce il testo o annulla l'operazione.

Modello di dominio

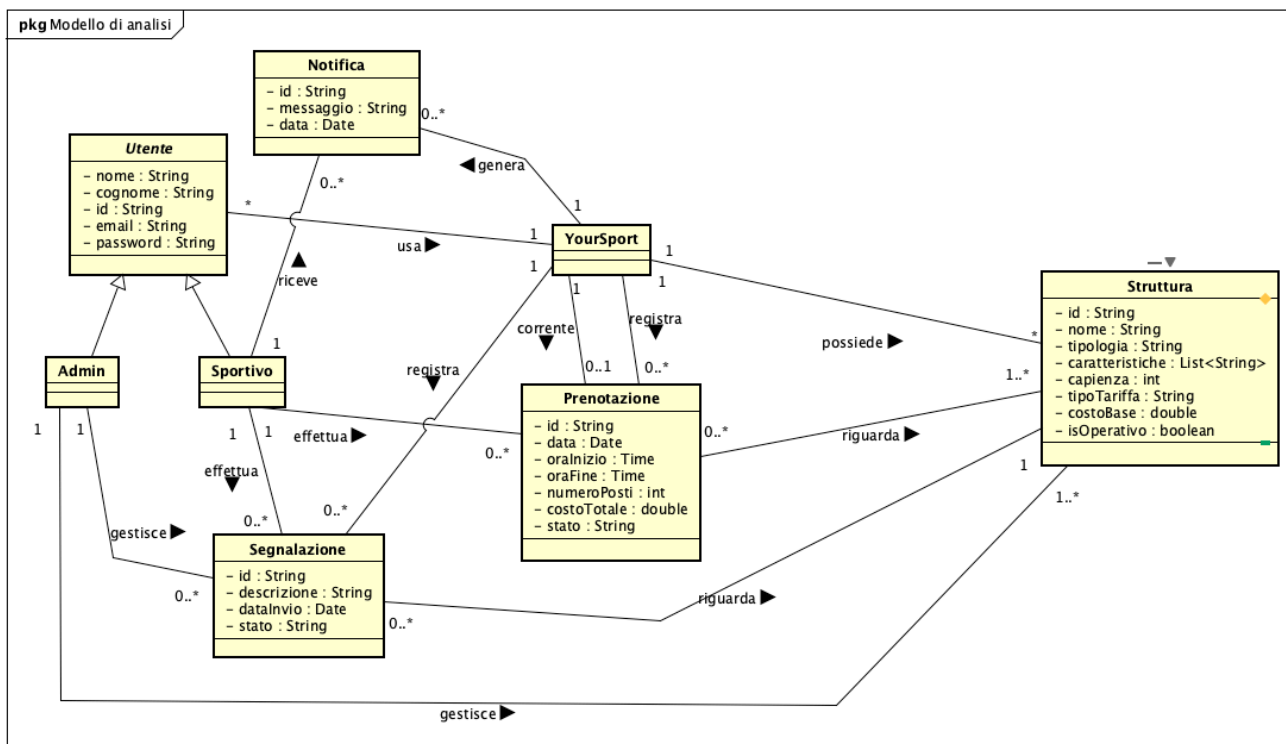
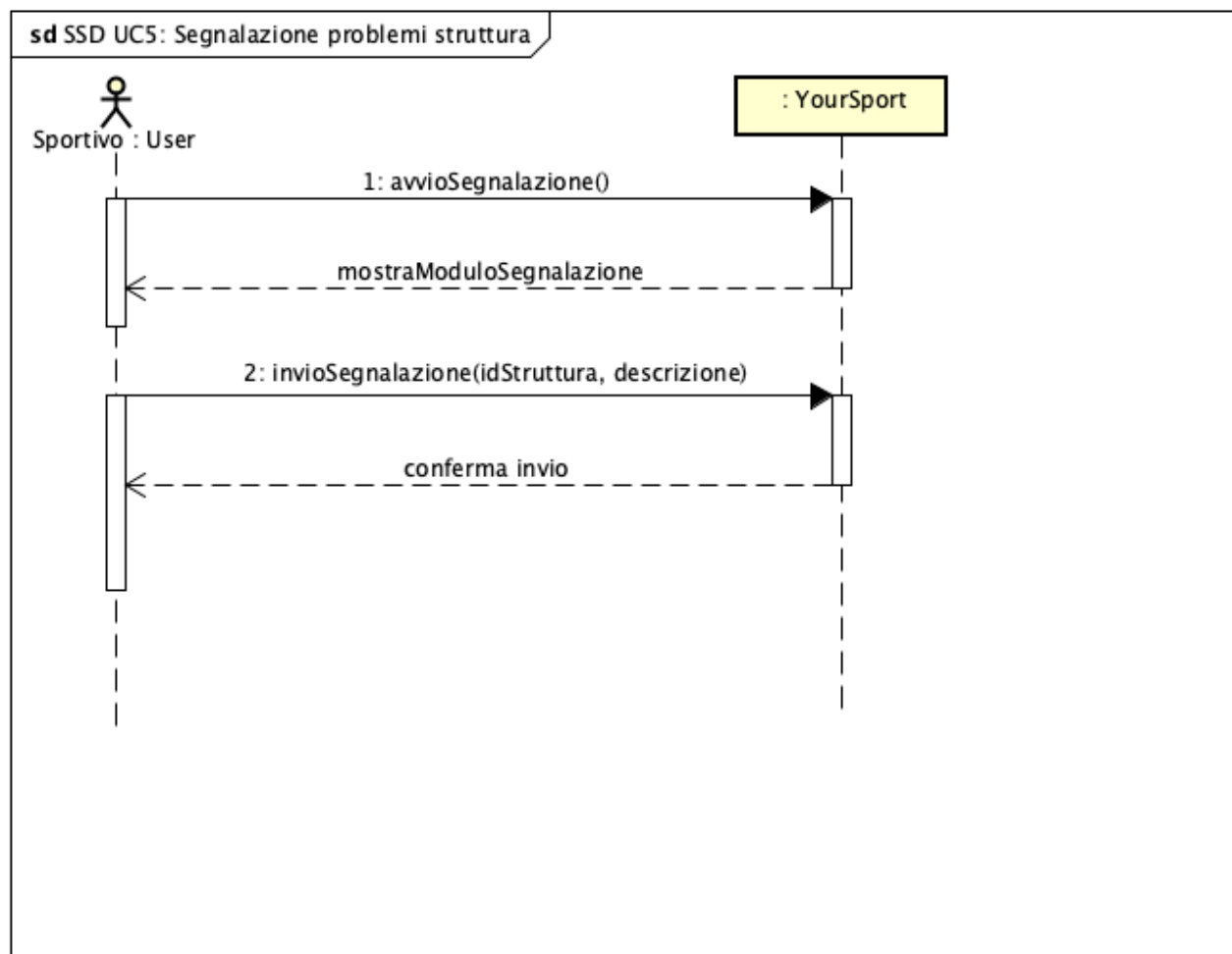


Diagramma di sequenza di sistema



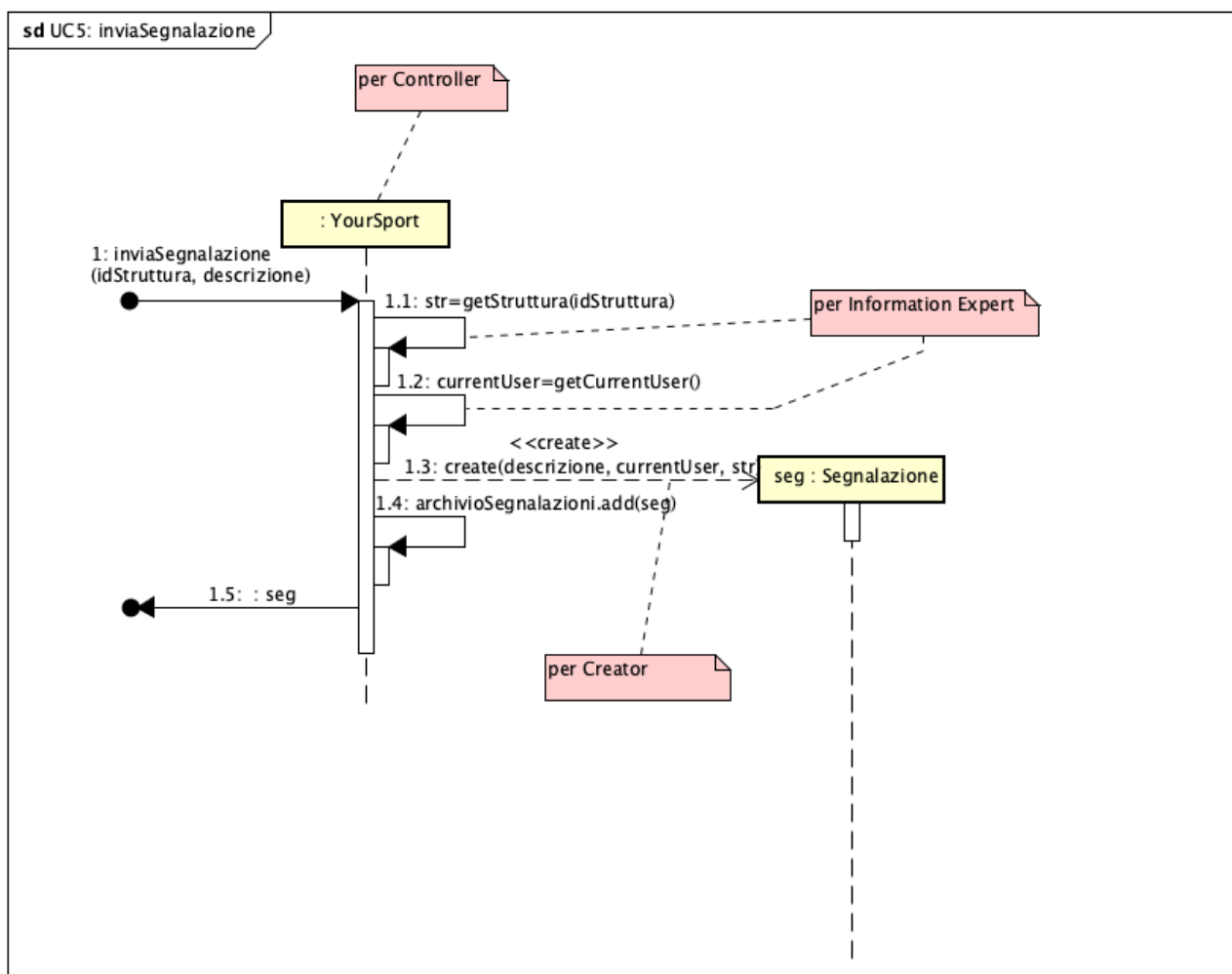
Contratti delle operazioni

-UC5

Operazioni	inviaSegnalazione(idStruttura: String, descrizione: String)
Riferimenti	UC5: Segnalazione problemi struttura
Pre-condizioni	-Lo Sportivo corrente è autenticato nel sistema. - Esiste un'istanza di Struttura il cui identificativo corrisponde a idStruttura

Post-condizioni	- È stata creata una nuova istanza <i>seg</i> della classe Segnalazione. - Gli attributi <i>descrizione</i>, <i>data</i> e <i>stato</i> dell'istanza <i>seg</i> sono stati inizializzati. - L'istanza <i>seg</i> è stata associata allo Sportivo corrente. - L'istanza <i>seg</i> è stata associata alla Struttura corrispondente.
-----------------	---

Diagrammi di sequenza



Testing

Test 1: Invia Segnalazione con Successo

Descrizione: Simuliamo l'invio di un ticket di segnalazione guasto (es. "Rete da tennis strappata") per una struttura realmente esistente, da parte di uno sportivo.

Scopo: Verificare che il sistema istanzi correttamente l'oggetto Segnalazione, assegnandogli un ID, la data odierna e impostando lo stato iniziale su "Aperta". Verifica inoltre le associazioni modellate nel DCD.

Realizzazione: Il test invoca il metodo `inviaSegnalazione` e, tramite asserzioni (`assertNotNull` e `assertEquals`), ispeziona l'oggetto restituito. Naviga le relazioni dell'oggetto per certificare che i puntatori leghino esattamente l'utente loggato e la struttura indicata.

Test 2: Errore Invia Segnalazione (Struttura Inesistente)

Descrizione: Simuliamo un errore in cui l'utente tenta di segnalare un guasto inserendo l'ID di un campo sportivo che non esiste nel database (es. "S_INVENTATO_99").

Scopo: Garantire l'integrità dei dati. Il sistema deve validare l'input e rifiutare la creazione del ticket se non può collegarlo a una struttura reale.

Realizzazione: Il test passa un ID falso al controller. Tramite `assertThrows`, si intercetta la `IllegalArgumentException` prevista. L'asserzione ispeziona il messaggio dell'eccezione per confermare che il rifiuto sia scattato perché la struttura non è stata trovata nell'archivio.

UC6: Gestione segnalazioni.

Nome caso d'uso	UC6: Gestione segnalazioni
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Admin
Parti interessate e interessi	Admin: vuole monitorare lo stato dei messaggi di segnalazione
Pre-condizioni	L'Admin è autenticato
Garanzie di successo	Lo stato della segnalazione viene aggiornato correttamente

Scenario principale di successo	1. L'Admin accede al pannello "Centro Segnalazioni". 2. Il Sistema mostra l'elenco di tutte le segnalazioni attive. 3. L'Admin seleziona una specifica segnalazione per visualizzarne i dettagli (Autore, Struttura, Descrizione). 4. L'Admin aggiorna lo stato della segnalazione (es. da "Aperta" a "In Lavorazione" o "Risolta"). 5. Il Sistema salva la modifica e aggiorna la visualizzazione. 6. L'Admin indica di aver terminato la gestione.
Estensioni	*a. In qualsiasi momento il sistema fallisce. 1. L'admin riavvia il software e ripristina lo stato. 2a. Non sono presenti segnalazioni nel sistema. 1. Il Sistema mostra un messaggio informativo ("Nessuna segnalazione presente").

Modello di dominio

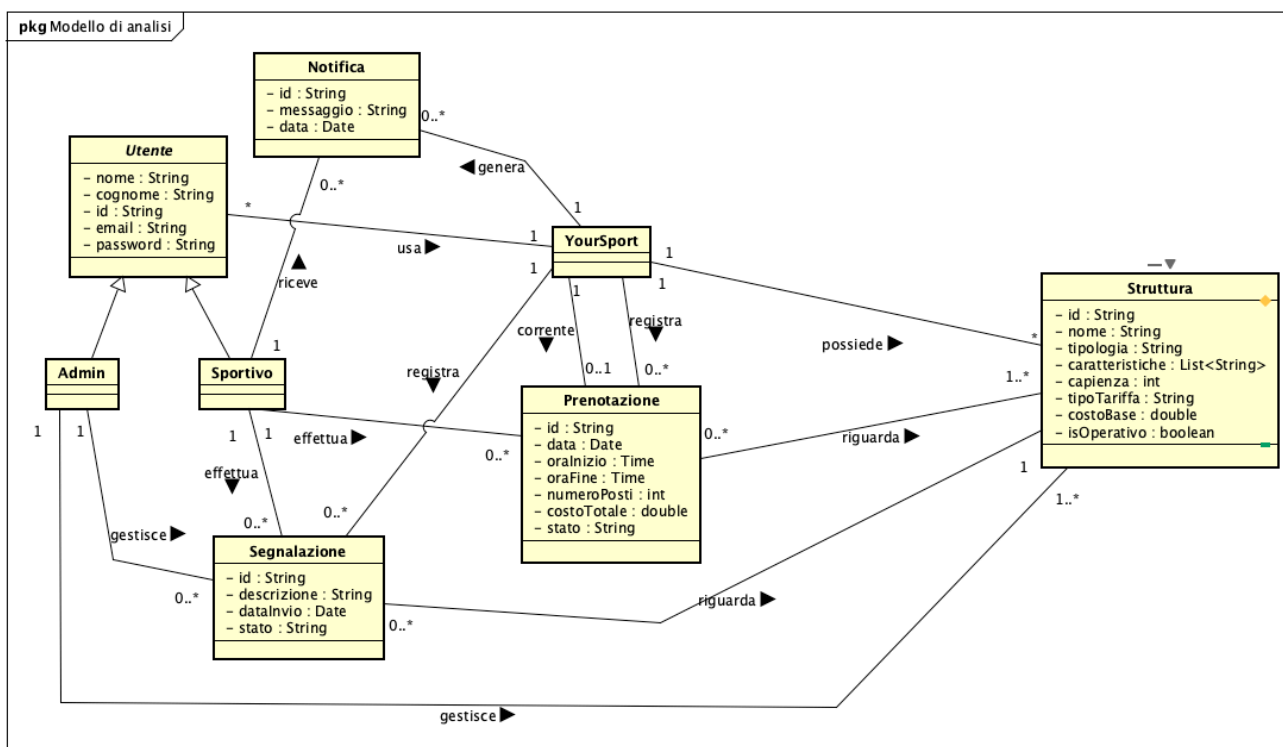
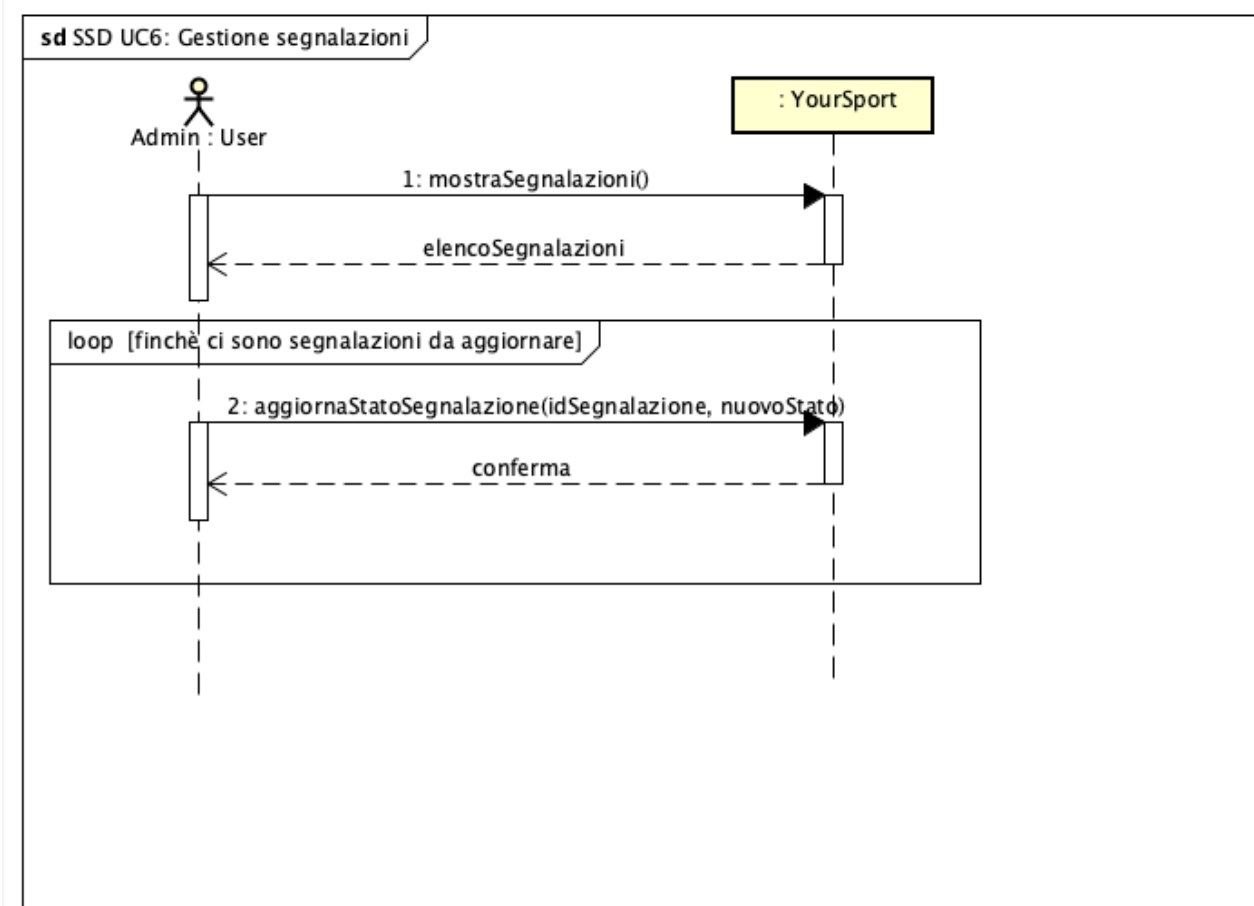


Diagramma di sequenza di sistema

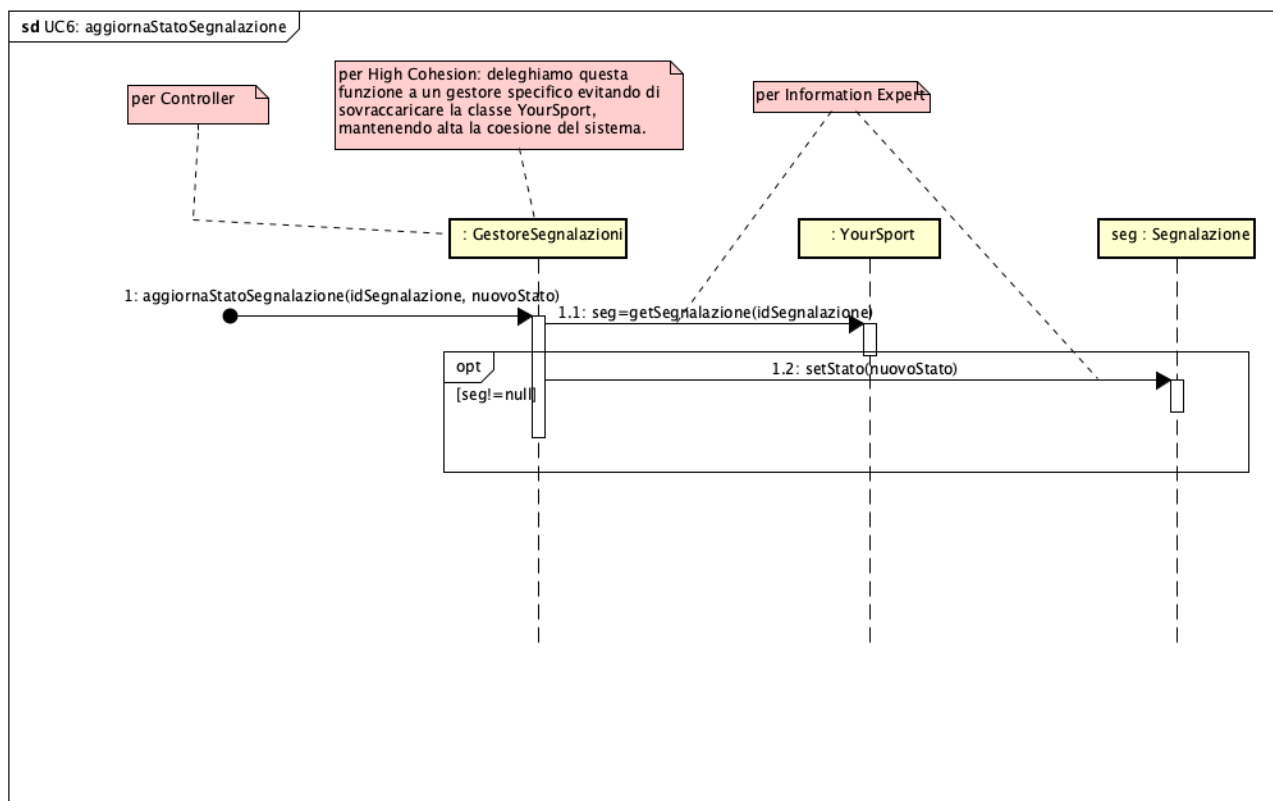


Contratti delle operazioni

-UC6

Operazioni	aggiornaStatoSegnalazione(idSegnalazione : String, nuovoStato: String)
Riferimenti	UC6: Gestione segnalazioni
Pre-condizioni	- L'utente corrente è autenticato nel sistema come Admin. - Esiste un'istanza di Segnalazione il cui identificativo corrisponde a idSegnalazione
Post-condizioni	- L'attributo <i>stato</i> dell'istanza Segnalazione corrispondente è stato modificato e impostato al valore del parametro nuovoStato.

Diagrammi di sequenza



Testing

Test 1: Lettura dell'Elenco Segnalazioni (Visualizzazione)

Descrizione: Simuliamo la richiesta dell'Amministratore di visualizzare a schermo l'intero storico dei guasti segnalati dagli sportivi.

Scopo: Dimostrare che il sistema riesca a recuperare correttamente l'archivio globale delle segnalazioni, assicurandosi che la connessione ai dati funzioni e che non vengano restituite liste nulle.

Realizzazione: Il test invoca il metodo `mostraSegnalazioni()`. Attraverso una serie di asserzioni (`assertNotNull`, `assertFalse`), verifica che la lista esista e non sia vuota. Successivamente, utilizza un ciclo `for` per ispezionare l'elenco e certificare (`assertTrue`) che la specifica segnalazione creata durante il setup iniziale del test sia fisicamente e correttamente presente al suo interno.

Test 2: Aggiornamento Stato Segnalazione (Successo)

Descrizione: Simuliamo l'intervento dell'Amministratore che prende in carico un ticket esistente, modificandone lo stato (ad esempio, passando da "Aperta" a "In risoluzione").

Scopo: Verificare il corretto funzionamento della logica di aggiornamento (il "cammino felice"). Il sistema deve individuare la segnalazione corretta e autorizzarla a sovrascrivere il proprio stato interno.

Realizzazione: Dopo un controllo di sicurezza per garantire che lo stato di partenza sia "Aperta", il test chiama il metodo `aggiornaStatoSegnalazione(...)` con parametri validi. La validazione finale si divide in due parti: prima si verifica che il metodo confermi il successo dell'operazione restituendo `true`; poi, ispezionando direttamente l'oggetto in memoria (`assertEquals`), si certifica che il suo attributo di stato sia stato effettivamente rimpiazzato con il nuovo valore.

Test 3: Aggiornamento Fallito per ID Inesistente (Riquadro [opt])

Descrizione: Simuliamo un errore di input da parte dell'Admin, che tenta di aggiornare lo stato di una segnalazione digitando un ID non presente a sistema (es. "ID_FALSO_INVENTATO_999").

Scopo: Testare la robustezza del sistema e l'esatta esecuzione del blocco opzionale (opt) modellato nell'UML. Se l'ID non viene trovato, il flusso deve interrompersi senza generare crash e senza corrompere i dati di altre segnalazioni.

Realizzazione: Il test passa deliberatamente un ID falso al controller. Tramite un'asserzione `assertFalse`, verifica che il metodo intercetti il problema e respinga la modifica. Infine, esegue un controllo di sicurezza cruciale sull'oggetto originale: assicura che il suo stato sia rimasto intatto ("Aperta"), dimostrando che il sistema ha protetto i dati da sovrascritture accidentali.

UC7: Sospensione e Riattivazione Struttura.

Nome caso d'uso	UC7: Sospensione e Riattivazione Struttura
Portata	Applicazione YourSport
Livello	Obiettivo utente
Attore primario	Admin
Parti interessate e interessi	Admin: deve chiudere una struttura e gestire i conflitti con le prenotazioni esistenti.
Pre-condizioni	L'Admin ha rilevato un guasto critico (tramite UC6)
Garanzie di successo	La struttura viene resa non operativa, le prenotazioni future impattate vengono annullate e i relativi sportivi vengono avvisati.
Scenario principale di successo	1. L'Admin seleziona la struttura da dichiarare fuori servizio. 2. Il Sistema richiede conferma per la sospensione dell'operatività. 3. L'Admin conferma la sospensione. 4. Il Sistema imposta l'attributo "isOperativo" della struttura a <i>False</i>. 5. Il Sistema identifica automaticamente tutte le prenotazioni future associate a quella struttura. 6. Il Sistema annulla forzatamente le prenotazioni trovate. 7. Il Sistema genera una notifica automatica (UC8) per ogni sportivo coinvolto spiegando il motivo del disservizio. 8. Il Sistema conferma all'Admin il numero di prenotazioni annullate e i messaggi inviati.

Estensioni

***a.** In qualsiasi momento il sistema fallisce.

1. L'admin riavvia il software e ripristina lo stato.

3a. L'Admin seleziona una struttura che è attualmente nello stato "Fuori Servizio".

1. Il Sistema chiede conferma per la riattivazione.
2. L'Admin conferma.
3. Il Sistema imposta lo stato della struttura come "Operativa", rendendola nuovamente disponibile per le prenotazioni (UC2).

5a. Non ci sono prenotazioni future per quella struttura.

1. Il Sistema disattiva semplicemente l'operatività della struttura e termina lo scenario con successo.

Modello di dominio

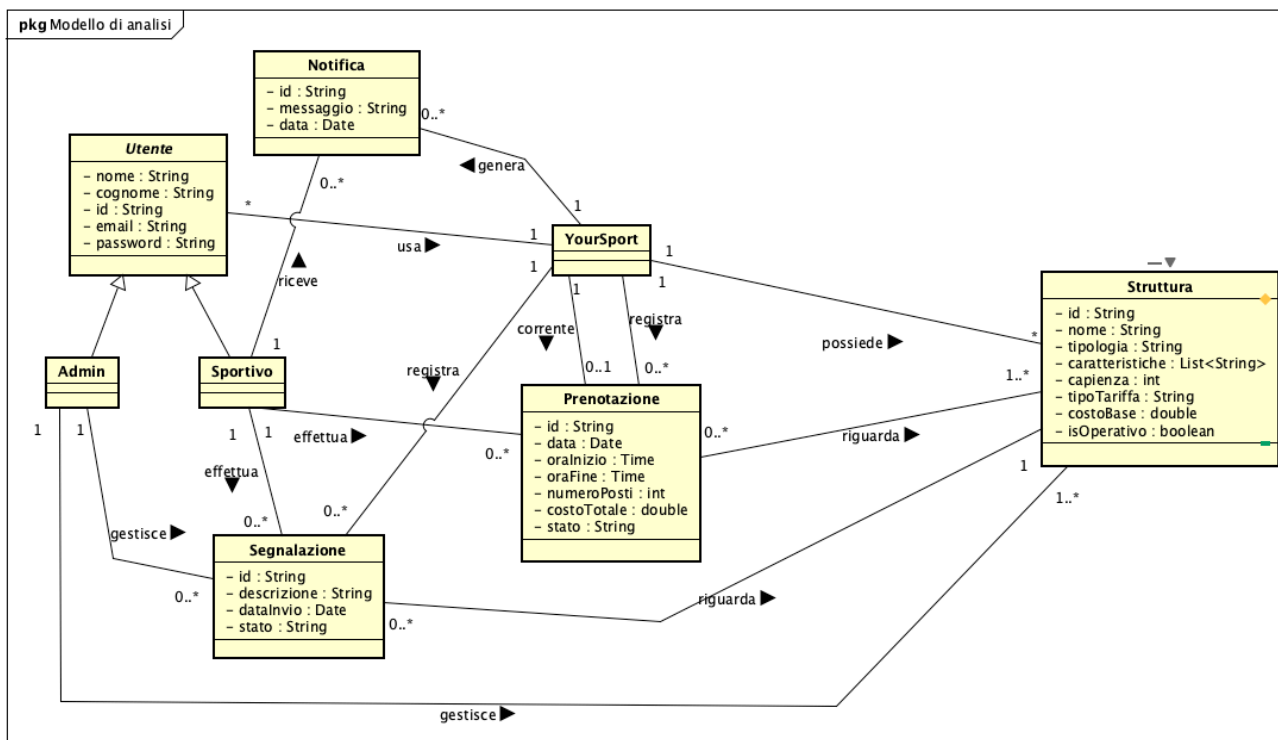
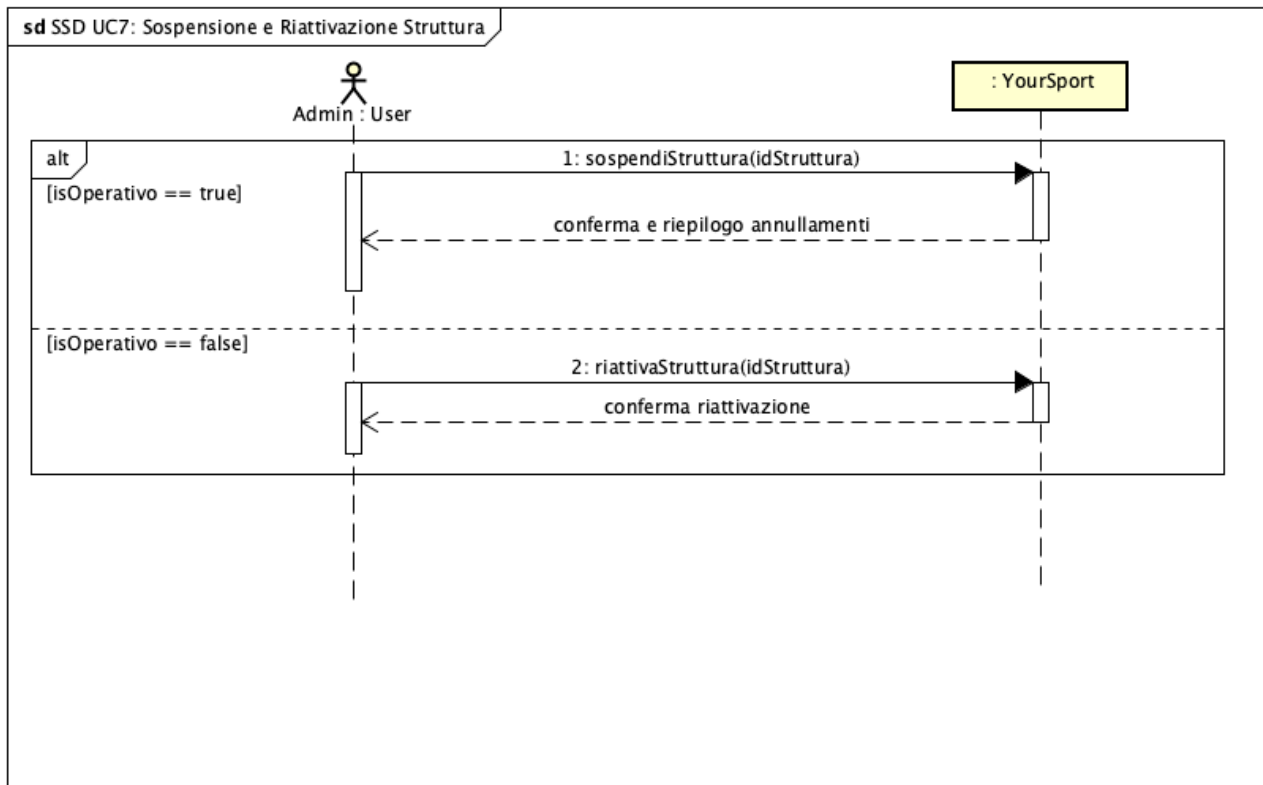


Diagramma di sequenza di sistema



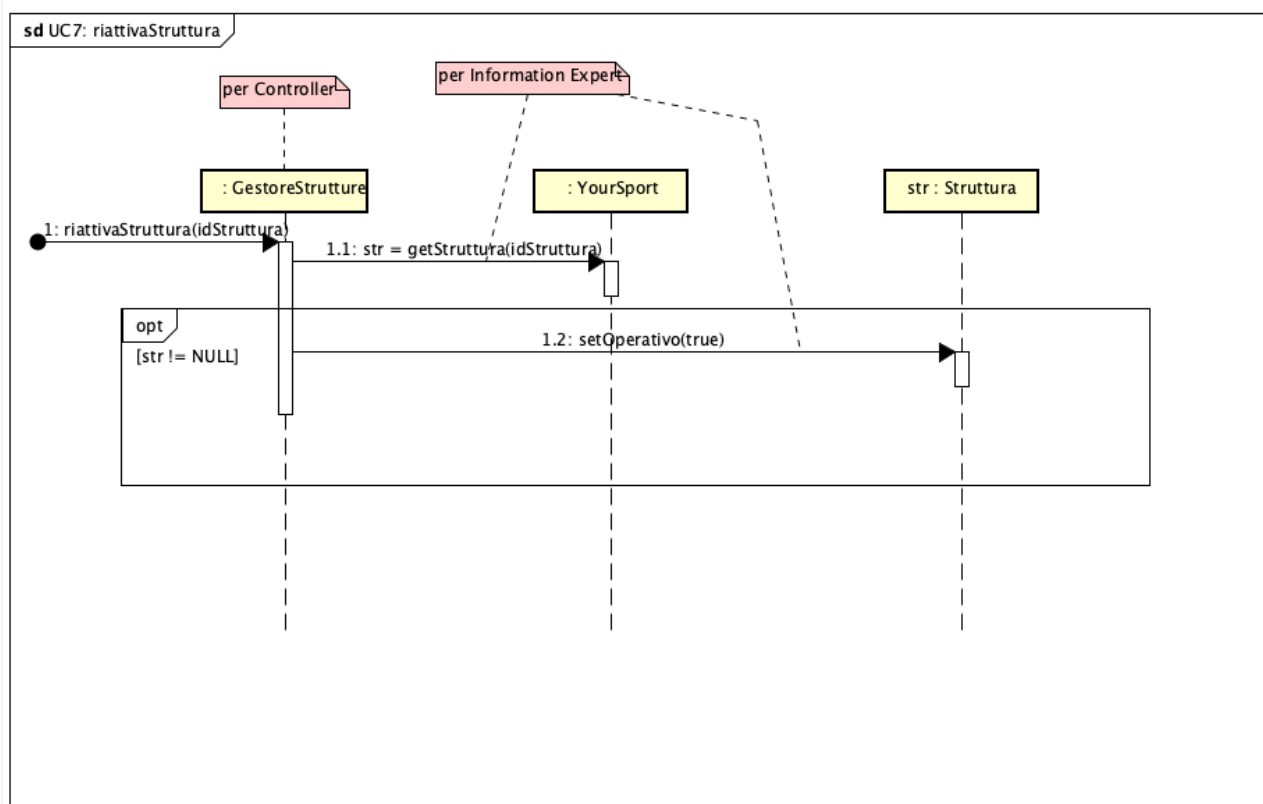
Contratti delle operazioni

Operazioni	sospendiStruttura(idStruttura: String)
Riferimenti	UC7: Sospensione e Riattivazione Struttura.
Pre-condizioni	- L'utente corrente è autenticato nel sistema come Admin. - Esiste un'istanza di Struttura il cui identificativo corrisponde a idStruttura.
Post-condizioni	- L'attributo <i>isOperativo</i> dell'istanza Struttura corrispondente è stato impostato a <i>false</i>. - L'attributo <i>stato</i> di tutte le istanze di Prenotazione future associate a tale Struttura è stato modificato in "Annullata". - È stata creata una nuova istanza di Notifica per ogni Sportivo la cui prenotazione è stata annullata forzatamente.

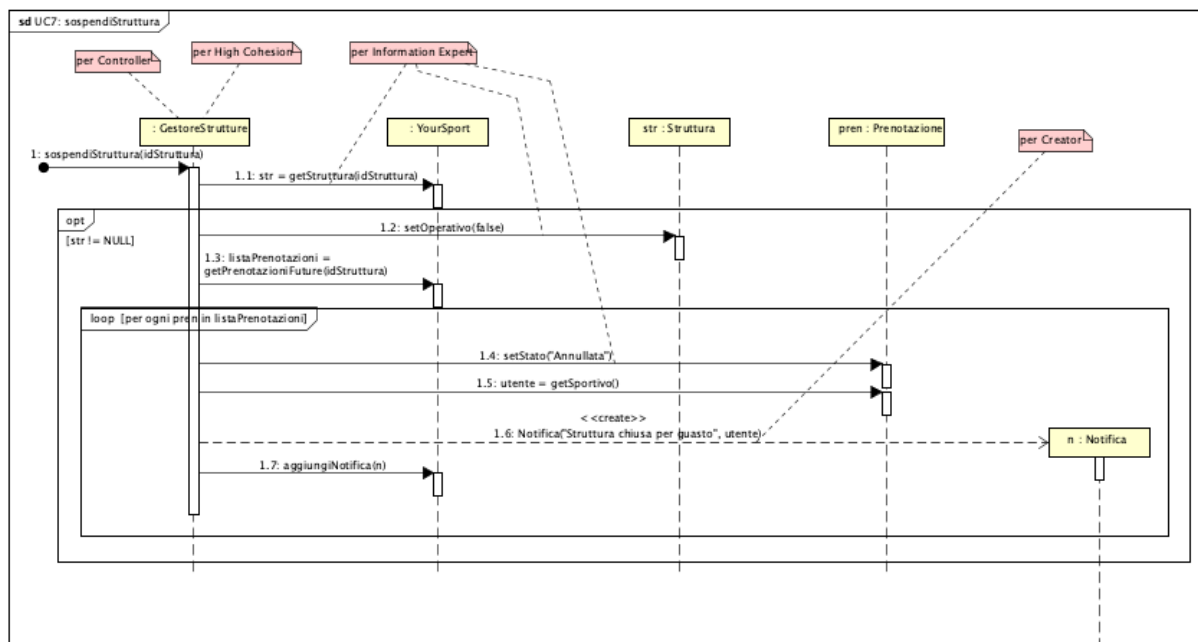
Operazioni	riattivaStruttura(idStruttura: String)
Riferimenti	UC7: Sospensione e Riattivazione Struttura.
Pre-condizioni	Esiste un'istanza di Struttura s associata all'identificatore idStruttura, e il suo attributo isOperativo è attualmente false.
Post-condizioni	L'attributo isOperativo dell'istanza s è stato modificato a true.

Diagrammi di sequenza

-riattivaStruttura



-sospendiStruttura



Testing

Test 1: Sospensione Struttura con Successo

Descrizione: Simuliamo l'azione dell'Amministratore che decide di sospendere (mettere fuori servizio) una struttura sportiva esistente (es. la struttura "S1").

Scopo: Verificare l'esatta esecuzione dell'intero flusso: la disattivazione fisica della struttura, l'annullamento di tutte le prenotazioni future associate e la generazione automatica delle notifiche per avvisare gli sportivi coinvolti dal disservizio.

Realizzazione: Il test invoca il metodo sospendiStruttura. Oltre a confermare l'esito positivo (assertTrue), esegue tre controlli di stato incrociati: 1) accerta che l'attributo di operatività della struttura sia diventato false; 2) verifica che lo stato di una prenotazione futura di test sia passato ad "Annullata"; 3) ispeziona l'archivio notifiche globale per assicurarsi che il messaggio di avviso sia stato creato e associato al corretto destinatario (messaggi 1.6 e 1.7 dell'SD).

Test 2: Sospensione Fallita per ID Inesistente (Blocco opt)

Descrizione: Simuliamo un errore di input in cui l'Admin tenta di sospendere una struttura fornendo un id non presente a sistema (es. "ID_FALSO_999").

Scopo: Testare la solidità del sistema. Il controller deve intercettare l'anomalia (bloccando l'esecuzione del riquadro opt dell'UML).

Realizzazione: Passando l'ID inesistente, il test verifica (assertFalse) che il metodo respinga l'operazione, dopo effettua un controllo di sicurezza per garantire che le strutture valide (come la "S1") siano rimaste regolarmente operative.

Test 3: Riattivazione Struttura con Successo (Scenario Alternativo)

Descrizione: Simuliamo l'operazione di ripristino, in cui l'Amministratore riaccende una struttura precedentemente posta fuori servizio, rendendola nuovamente disponibile.

Scopo: Dimostrare che il sistema gestisca correttamente il cambio di stato inverso, rendendo la risorsa di nuovo interfacciabile con i flussi di prenotazione degli sportivi.

Realizzazione: Il test prepara l'ambiente forzando manualmente lo spegnimento di una struttura di test ("S2"). Successivamente, invoca riattivaStruttura e certifica tramite asserzioni che il metodo completi l'operazione con successo e che l'attributo interno operativo della struttura sia tornato a true.

Test 4: Riattivazione Fallita per ID Inesistente

Descrizione: Simuliamo il tentativo di riattivare una struttura fornendo un ID finto ("STRUTTURA_GHOST").

Scopo: Assicurare che anche il flusso di riattivazione includa la validazione dei dati e sia protetto da input errati o malformati.

Realizzazione: Il test passa il parametro errato al gestore e verifica (assertFalse) che il sistema valuti negativamente la condizione iniziale e respinga la richiesta.